

tCLD-Net: A Transfer Learning Internet Encrypted Traffic Classification Scheme Based on Convolution Neural Network and Long Short-Term Memory Network

Xinyi Hu^{1*†}, *Chunxiang Gu^{2*†}, Yihang Chen^{3*} and Fushan Wei^{4*†} *State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, China

[†]Henan Key Laboratory of Network Cryptography Technology, Zhengzhou, China

Email: ¹huxinyi.1994@foxmail.com, ²gcx5209@126.com, ³cyhpaper@163.com, ⁴weifs831020@163.com

Abstract—The Internet is about to enter the era of full encryption. Traditional traffic classification methods only work well in non-encrypted environments. How to identify the specific types of network encrypted traffic in a non-encrypted environment without decryption is one of the foundations for maintaining cyberspace security. Traffic classification based on machine learning relies heavily on the prior knowledge of experts to construct feature sets. Although traffic classification based on deep learning can reduce human intervention, it requires a large amount of labeled data for parameter determination. This paper proposes a tCLD-Net model that combines transfer learning and deep learning. It can be trained on a small amount of labeled data to distinguish network encrypted traffic with a high accuracy. It pre-trains a CLD-Net model in the source domain data set, and fixes the parameters of the convolutional neural network module in it, and trains and tests it in the target domain data set. In order to verify the effectiveness of the tCLD-Net model, we use the ISCX public data set to conduct experiments. The results show that our proposed model can complete 100 epochs training in 208 seconds when the training set only occupies 20% of the target domain. And achieve a classification accuracy rate about 86%. This is 4% higher than the model without pre-training, and the training time is only one third of the model without pre-training.

Index Terms—Deep Learning, Transfer Learning, Pre-training, Internet Traffic, Encrypted Traffic, Traffic Classification

I. INTRODUCTION

Network traffic classification and identification is an important foundation for network monitoring and management, and it is also one of the key technologies for maintaining cyberspace security. However, with the development of network encryption, the existing port-based and deep packet inspection-based technologies are difficult to identify and classify network traffic, which causes difficulties in traffic review. Researchers began to consider introducing machine learning technology into the field of traffic analysis. According to prior experience, feature engineering is carried out, feature set is constructed, and then the feature set is input into the machine learning classifier for classification and identification.

Since traditional machine learning methods need to manually construct feature sets, the quality of the feature set

directly affects the effect of the classifier. In order to reduce manual intervention, researchers use deep learning technology to automatically perform feature learning and classification. As the representative of deep learning technology, artificial neural network is regarded as an end-to-end model. Researchers only need to input the raw flows, or simply process the raw flows, and then input them into the neural network, the neural network will directly output the results that researchers want.

However, deep learning models require a large amount of high-quality labeled data, and in the real environment, there are often only a small amount of labeled data or even unlabeled data. A small amount of data cannot be solved by deep learning because there are too many parameters and the number of samples cannot support it to learn well, which can easily lead to overfitting of the model. Transfer learning can transfer knowledge from existing data to help future learning. The goal of transfer learning is to use knowledge learned from one environment to help learning tasks in a new environment. Considering that data or tasks are related, this paper combines deep learning and transfer learning to propose the tCLD-Net model for network encryption traffic classification. The model that has been trained on the source domain is used as a pre-training model and passed to the target domain for training and testing. This can speed up the optimization of the learning efficiency of the model, reduce the amount of sample data required, and reduce training costs.

The rest of the paper is organized as follows. Section 2 summarizes related work. Section 3 describes the problem definition and introduces the proposed model. Section 4 gives the data set and experimental results. Section 5 summarizes the paper and discusses future work.

II. RELATED WORK

A. Traffic Analysis Using Deep Learning

Researches on the use of deep learning for traffic analysis began to appear in 2015 [1], and has developed rapidly in recent years [2]–[4], [6].

In 2018, Tong et al. [2] proposed a new technology of traffic classification based on convolutional neural network

for Google's QUIC protocol (Quick UDP Internet Connection protocol). This method can distinguish Google Hangout Chat, Google Hangout Voice Call, YouTube, File Transfer and Google Play Music with a micro average F1-score of 99.24%.

In 2019, Aceto et al. [3] proposed a multimode deep learning framework for encrypted traffic classification, which can utilize the heterogeneity of traffic data to overcome the existing single-mode deep learning-based traffic classification performance limitation, and verified it on the artificially generated mobile encrypted traffic dataset.

In 2020, Bu et al. [4] proposed a neural network model with a deeply parallel network-in-network (NIN) structure to classify encrypted network traffic. The experimental results on the public dataset ISCX VPN-nonVPN [5] (Virtual Private Network) show that compared with the traditional Convolution Neural Network (CNN), the NIN model can achieve a better balance between classification accuracy and model complexity.

In 2021, Hu et al. [6] proposed a network encryption traffic classification model CLD-Net. The model combines CNN and Long Short-Term Memory (LSTM) network to perform neural network feature learning from both statistics and time series, and changes the number of channels to make the model more suitable for the reorganized data structure, thereby improving classification performance. CLD-Net has higher accuracy than other methods when encrypting traffic classification for public network traffic data, and the average accuracy of the eight-class is 92.89%.

B. Traffic Analysis Using Transfer Learning

Transfer learning can eliminate the assumption that the training set and the test set must be independent and identically distributed. Therefore, as deep learning is developing in the field of traffic analysis, researchers have also begun to use transfer learning for traffic analysis [7], [9], [11]–[14].

In 2018, Sun et al. [7] introduced transfer learning to the task of traffic classification. The maximum entropy model Maxent is used as the basic classifier, and TrAdaBoost [8] transfer learning model is selected. The comparison with traditional machine learning method shows that the proposed method can improve overall performance.

In 2019, Niu et al. [11] proposed a network intrusion detection method based on transfer component analysis. Through domain adaptation, differently distributed data sets are mapped to the same subspace. The model uses the basic classifier in the shared subspace for training and detects new traffic data generated from different domains.

In 2020, Han et al. [12] transferred the pre-trained language model with language data to the network traffic classification problem, then fine-tune the model with labeled network traffic. This method has faster convergence speed, higher accuracy rate and lower false positive rate.

In 2021, Dhillon et al. [13], [14] used various deep learning methods to build a network intrusion detection system (NIDS), and used deep transfer learning technology to further improve the program to make it effective in the real environment. The knowledge learned by the model in the source domain with a large number of computing and data resources is transferred

to the target domain with sparse availability. The experimental results on the public dataset USNW-15 show that this method can improve the overall effectiveness of the intrusion detection system (IDS) in the real environment.

III. PROPOSED METHODOLOGY

Before the data is input to the model, the raw data must be preprocessed. See Algorithm 1 for details. The preprocessing procedure mainly includes 5 steps:

- 1) Lines 2-4 are traffic split. According to the size of the flow, the raw data in the data set are randomly split into flow fragments containing 10 consecutive data packets, and the training set and the test set are divided proportionally.
- 2) Line 7 is traffic cleaning, only read the payload part of the packet, and the uniform length is 256 bytes.
- 3) Lines 8-10 are traffic reorganization, where the 256-byte payload is divided into different bytes and then reorganized.
- 4) Line 11 is traffic conversion. The 5 sequences after step 2 and step 3 are converted to decimal by byte to obtain a grayscale image.
- 5) Lines 15-19 are insertion time interval. According to the length of the arrival time interval of two adjacent packets, with 1 second as the limit, a matrix of all 1s is selected as the time interval information.

After the preprocessing, the obtained data packet matrices are input into the model. In this paper, we combine the CLD-Net model [9] with transfer learning to obtain an improved tCLD-Net model, as shown in Figure 1.

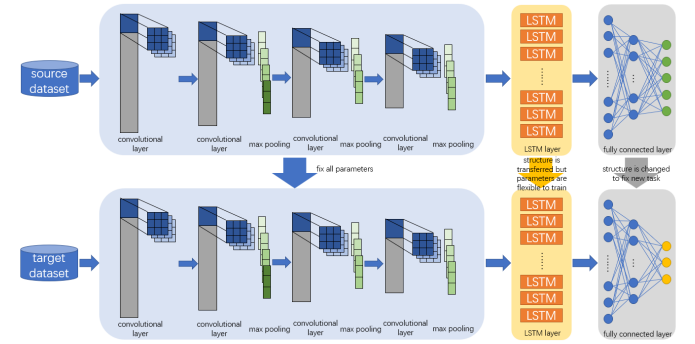


Fig. 1. The tCLD-Net model structure. The upper layer is the pre-training process, and the lower layer is the transfer process.

The pre-training process is performed in the source domain using the CLD-Net model. The CLD-Net model includes a CNN module, an LSTM module, and a Dense module. The CNN module contains 10 1D-CNNs (one-dimensional CNNs), each CNN contains 4 convolutional layers, and the last 3 convolutional layers are connected to a maximum pooling layer. The LSTM module contains 10 LSTMs. The Dense module contains 3 fully connected layers. After the source domain data is preprocessed, it is converted into a grayscale image format, and the CNN's ability to process image data is used to automatically learn features. After passing through the LSTM module, the ability of LSTM to process

Algorithm 1 Preprocessing Algorithm.

Input: Raw dataset D ; Number of traffic classes C ;**Output:** Packet matrices set $\mathbf{P}$;

```
1: for each  $i \in [1, C]$  do
2:   Randomly select  $N$  consecutive 10 packets in  $D$ ;
3:   10 consecutive packets consist a flow  $F$ , that is, the number of  $F$  is  $N$ ;
4:   Randomly divide the  $F$  into a training set  $F^{train}$  and a testing set  $F^{test}$  in proportion;
5:   for each  $j \in [1, N]$  do
6:     for each packet  $p \in F$  do
7:       Trim  $p$  and uniform length of 256 bytes,  $p \rightarrow$  payload  $\mathbf{p}_{256} = (b_1, b_2, \dots, b_{8 \times 256})$ ;
8:       256 bytes are divided into 64 bytes, 32 bytes, 16 bytes and 8 bytes to get the packet sequence  $\mathbf{p}_{256} \rightarrow \mathbf{p}_\alpha =$ 
        $(b_{1,1}^\alpha, \dots, b_{1,\alpha \times 8}^\alpha, \dots, b_{\frac{256}{8},1}^\alpha, \dots, b_{\frac{256}{8},8\alpha}^\alpha)$ ;
9:       Separate 256 bytes by the length of 1 byte;
10:      Extract each group of the same bits in turn as the recombined part to obtain recombination sequence  $\mathbf{p}_\alpha \rightarrow$ 
 $\hat{\mathbf{p}}_\alpha = (b_{1,1}^\alpha, \dots, b_{\frac{256}{8},1}^\alpha, \dots, b_{1,8\alpha}^\alpha, \dots, b_{\frac{256}{8},8\alpha}^\alpha)$ ;
11:      Generate the grayscale image format to get the packet matrix,  $\hat{\mathbf{p}}_\alpha \rightarrow P, P^T = (\mathbf{p}_{256}, \hat{\mathbf{p}}_{64}, \hat{\mathbf{p}}_{32}, \hat{\mathbf{p}}_{16}, \hat{\mathbf{p}}_8)$ ;
12:    end for;
13:  end for;
14:  for  $m = 1$  to  $10N - 1$  do
15:    Count the time interval of  $P_m$  and  $P_{m+1}$ ;
16:    if the time interval between  $P_m$  and  $P_{m+1} < 1s$  then
17:      Input  $P_{m+1}$  after  $P_m$  to the model;
18:    else
19:      Add  $P_0 = \begin{pmatrix} 1 & \dots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 1 \end{pmatrix}_{5 \times 256}$  between  $P_m$  and  $P_{m+1}$ ;
20:    end if
21:  end for;
22: end for;
23: return Packet matrices set  $\mathbf{P}$ ;
```

long stream data is used to continue the feature mapping. Finally, through the Dense module, the fully connected layers are used to perform feature dimensionality reduction, and the final classification results are output. After pre-training for 100 epochs, the pre-trained model is obtained. See Algorithm 2 for the specific pre-training algorithm.

Fix the parameters of the CNN module in the pre-trained model, and transfer the model to the target domain to continue training and testing. After the transfer, the intersection of the target domain data and the source domain data is an empty set, and the amount of target domain data is reduced. During the 100 epoches training process, fine-tune the parameters of the LSTM module and the Dense module. Finally, the classification results are predicted in the test set of the target domain, and the results are evaluated.

IV. EXPERIMENT

A. Dataset

This paper uses the public data set ISCX VPN 2016 [5], and selects the traffic data of the two social software Facebook and Hangout as the experimental data. Facebook traffic includes three applications: chat, audio, and video, Hangout traffic includes two applications: chat and audio. And the traffic of each specific application can use VPN for protocol-encapsulated network traffic, or just ordinary network traffic nonVPN.

TABLE I
DATA CLASSES AND DATA VOLUME OF THE SOURCE AND TARGET DOMAINS.

	Source Dataset	Target Dataset
Data classes	facebook-audio, facebook-chat, facebook-video, vpn-facebook-audio, vpn-facebook-chat	hangout-chat, hangout-audio, vpn-hangout-audio
Number of flows	4059	6590

The experiment uses a total of 8 classes of data, 5 classes are selected as the source domain data, and the remaining 3 classes are used as the target domain data. See Table 1 for details. Among them, the target domain data randomly selects 20%, 40%, 60%, 80% of the data as the training set, and the rest as the test set.

B. Experiment Results

The experiment selects 4 models for training and testing of the target domain. They are: model without pre-training, CLD-Net pre-training model with fixed CNN module parameters, CLD-Net pre-training model with fixed CNN module parameters and LSTM module parameters, CLD-Net pre-training model without fixed parameters. The experiment chooses clas-

Algorithm 2 Pre-training Algorithm.**Input:** Source dataset D_S ; Number of traffic classes C_S ; Number of epoques Epo ;**Output:** Pre-trained model;

```

1: Preprocess  $D_S$  using Algorithm 1 to obtain packet matrices set;
2: for  $n \in [1, Epo]$  do
3:   for each  $P$  in Packet matrices set do
4:     Let  $P^T = (\mathbf{x}^1, \mathbf{x}^2, \mathbf{x}^3, \mathbf{x}^4, \mathbf{x}^5)$ ;
5:     for each  $\mathbf{x} \in P$  do
6:       Let  $W$  is the weight,  $B$  is the bias, the output of the first convolutional layer is  $Out_{ij}^1 = Relu(Conv_{ij}^1)$ , where
        $Conv_{ij}^1 = \sum_{m=1}^3 W_{m,j}^0 x_{i+m-1,j}^0 + B_j^0$ ;
7:       The output of the second convolutional layer is  $Out_{ij}^2 = Relu(\max_{r \leq 3}(Conv_{i \times 1+r,j}^2))$ , where  $Conv_{ij}^2 =$ 
        $\sum_{m=1}^M W_{m,j}^1 x_{i+m-1,j}^1 + B_j^1$ ;
8:       The third and forth layers are the same as the second layer, the output of CNN module output is  $Out_{ij}^4 =$ 
        $Relu(\max_{r \leq 3}(Conv_{i \times 1+r,j}^4))$ ; // CNN module;
9:     end for;
10:    Let  $X_t$  (the input of LSTM at time  $t$ ) pass through the forget gate  $F(t) = sigmoid(W_f^T S_{t-1} + U_f^T X_t + B_f)$  and
    output gate  $O(t) = sigmoid(W_o^T S_{t-1} + U_o^T X_t + B_o)$ , where  $U$  is the weight matrix of hidden state,  $C_{t-1}$  is the cell
    state at the previous time,  $S_{t-1}$  is the hidden state;
11:    The cell state at time  $t$  is  $C_t = C_{t-1} * F(t) + I(t) * R(t)$ , the output of the LSTM module is the hidden state at
    time  $t$ ,  $S_t = tanh(C_t) * O(t)$ ; // LSTM module;
12:    Set the output of the first fully connected layer to half of the input;
13:    Add a Dropout layer after the second Dense layer, and remove the training unit from the network according to a
    certain probability to prevent overfitting;
14:    Add a Softmax classifier after the third Dense layer, the data is divided into  $C_S$  classes; // Dense module;
15:  end for;
16: end for;
17: return Pre-trained model;

```

sification accuracy and training time as the basis for evaluating the model.

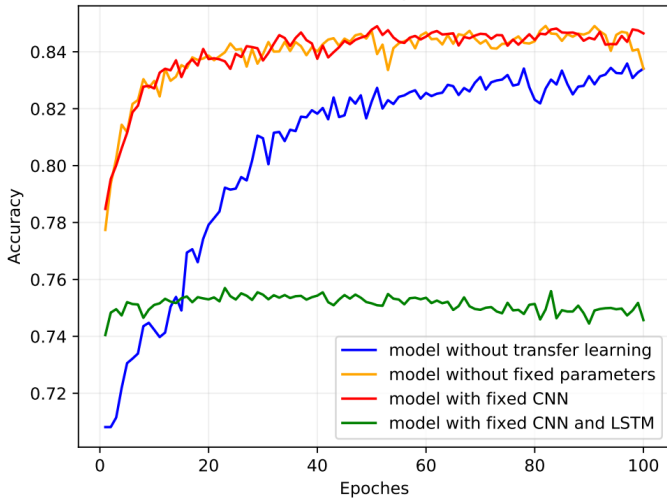


Fig. 2. Accuracy curve of 4 different models based on 20% random samples on target domain.

It can be seen from Figure 2 that when the training set is 20%, the training speed is slow, and the starting point accuracy is low. It is gradually improved by continuous iteration, and the highest accuracy rate can reach 82%. The CLD-Net pre-training model with fixed CNN module parameters and LSTM module parameters has a high starting point, but it can hardly be improved because of too many fixed parameters. The CLD-

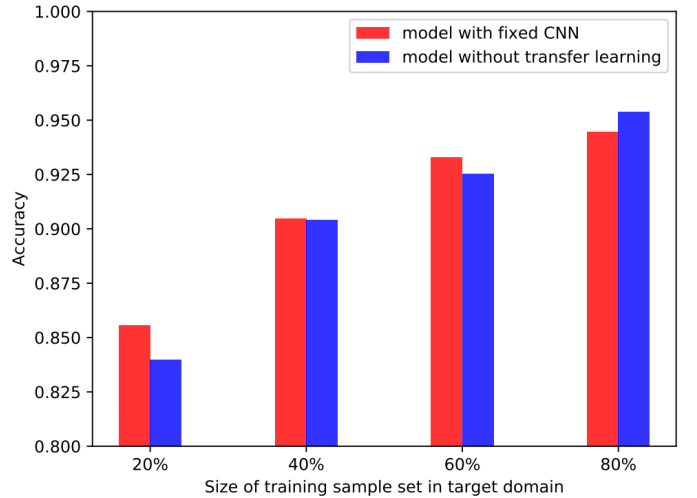


Fig. 3. Accuracy of the 2 models with and without transfer learning based on 20%, 40%, 60%, 80% random samples for training on target domain.

Net pre-training model without fixed parameters has a high starting point, but the training speed is slow, and later all parameters are fine-tuned, which will overfit because of too small training set. The best effect is the CLD-Net pre-training model with fixed CNN module parameters. The starting point is high, and because there are few parameters need to be fine-tuned, the training speed is fast, and accuracy rate 86% can be quickly achieved.

It can be seen from Figure 3 that when the training set

is 20%, there is a large gap between the model without pre-training and the CLD-Net pre-training model with fixed CNN module parameters, which is about 4%. When the training set is 40%, the gap is not much, about 1.6%. Eventually, as the size of the training set becomes larger, the advantages of transfer learning will gradually disappear.

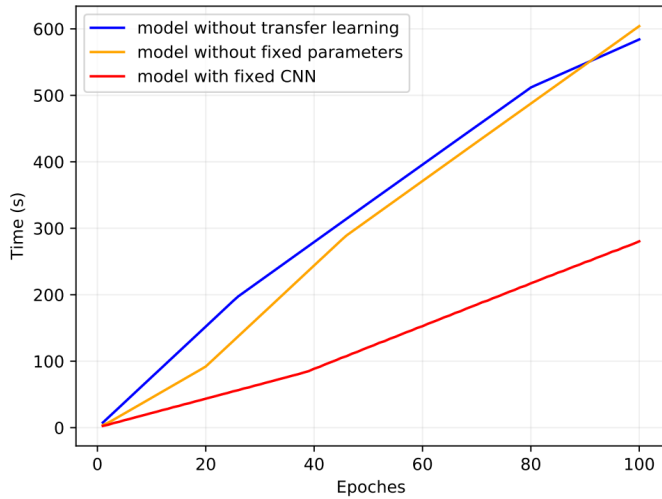


Fig. 4. Training time of 3 different models based on 20% random samples on target domain.

It can be seen from Figure 4 that the training time of model without pre-training is 584.1 seconds. The training time of the CLD-Net pre-training model without fixed parameters is 604.1 seconds. The training time of the CLD-Net pre-training model with fixed CNN module parameters is 208.4 seconds.

Therefore, transfer learning can effectively improve the classification accuracy and greatly reduce the amount of data required for training. When the training set is reduced to 20%, the CLD-Net pre-training model with fixed CNN module parameters can not only achieve a higher classification accuracy, but also requires a shorter training time, which is the best among the four models.

V. CONCLUSION

In this paper, we propose a network encryption traffic classification model based on transfer learning and deep learning. Compared with the traffic classification method based solely on deep learning, this model does not require a large amount of labeled data for training. In this model, through pre-training and fixing the parameters of the CNN module, the classification accuracy can be effectively improved and the training time can be greatly reduced. The experimental results on the public data set show that when the training set only accounts for 20% of the data set, the performance of the tCLD-Net model is significantly improved compared to the model without pre-training.

In future work, we can consider using the pre-trained model in the unlabeled data set to further fit the actual network environment. In addition, how to eliminate the impact of the unbalanced data set is also a future research direction.

ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China under Grant 61772548.

REFERENCES

- [1] Z. Wang, "The applications of deep learning on traffic identification," *BlackHat USA*, vol. 24, pp. 1–10, 2015.
- [2] V. Tong, H. A. Tran, S. Souihi, A. Mellouk, "A Novel QUIC Traffic Classifier Based on Convolutional Neural Networks," *IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, 2018.
- [3] G. Aceto, D. Ciunzo, A. Montieri, A. Pescapé, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Computer Networks*, vol. 165, 2019.
- [4] Z. Bu, B. Zhou, P. Cheng, K. Zhang, Z. Ling, "Encrypted Network Traffic Classification Using Deep and Parallel Network-in-Network Models," *IEEE Access*, vol. 8, pp. 132950–132959, 2020.
- [5] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic using Time-related Features," *Information Systems Security and Privacy (ICISSP)*, 2016.
- [6] X. Hu, C. Gu, F. Wei, "CLD-Net: A Network Combining CNN and LSTM for Internet Encrypted Traffic Classification," *Security and Communication Networks*, vol. 2021, 2021.
- [7] G. Sun, L. Liang, T. Chen, F. Xiao, F. Lang, "Network traffic classification based on transfer learning," *Computers and Electrical Engineering*, vol. 69, pp. 920–927, 2018.
- [8] W. Dai, Q. Yang, G. Xue, Y. Yu, "Boosting for transfer learning," *International Conference on Machine Learning (ICML)*, pp. 193–200, 2007.
- [9] S. Taheri, M. Salem, J. Yuan, "Leveraging Image Representation of Network Traffic Data and Transfer Learning in Botnet Detection," *Big Data Cognitive Computing*, vol. 2, pp. 37–52, 2018.
- [10] S. García, M. Grill, J. Stiborek, A. Zunino, "An empirical comparison of botnet detection methods," *Computer Security*, vol. 45, pp. 100–123, 2014.
- [11] J. Niu, Y. Zhang, D. Liu, D. Guo, Y. Teng, "Abnormal Network Traffic Detection Based on Transfer Component Analysis," *IEEE International Conference on Communications Workshops (ICC Workshops)*, pp. 1–6, 2019.
- [12] L. Han, X. Zeng, L. Song, "A Novel Transfer Learning Based on Albert For Malicious Network Traffic Classification," *International Journal of Innovative Computing, Information and Control*, vol. 16, pp. 2103–2119, 2020.
- [13] H. Dhillon, A. Haque, "Towards Network Traffic Monitoring Using Deep Transfer Learning," *IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 1089–1096, 2020.
- [14] H. Dhillon, "Building Effective Network Security Frameworks using Deep Transfer Learning Techniques," *Electronic Thesis and Dissertation Repository*, 2021.